

Instructions

- This is an individual programming assignment. Your code, experiments, results, analysis, and report must be your own.
- The assignment is due on **11 October 2026 via LMS**.
- All submitted code must be available in a public GitHub repository. Do not commit raw datasets, public model weights, or unnecessary large checkpoints.
- Submit an **8-page NeurIPS-style PDF report** presenting your experiments, results, and analysis. References do not count toward the page limit. Include the link to your GitHub repository at the end of the abstract.
- **AI Usage Policy:** You may not use generative AI to write any part of your PDF report. The report’s language, interpretation, and analysis must be entirely your own.
- **Coding Assistance:** You may use public libraries, built-in functions, open-source implementations, and coding assistance from an LLM. However, you are responsible for every submitted line of code and must understand how it works. Clearly attribute materially reused external code in your README.
- The released code is starter infrastructure rather than a reference solution. Tasks 1–3 contain incomplete training/ablation scaffolds and one deliberate algorithmic defect in each core objective implementation. Validate and correct the implementation against the mathematical objective before relying on its output.
- Fix random seeds where practical, record all reported hyperparameters, preserve the exact prompt IDs and data indices used, and save machine-readable logs for every result in the report.
- Read the complete assignment before beginning. **All experiments listed below are required.**

Overview & Motivation

Post-training changes a language model from a next-token predictor into a policy whose outputs are shaped by preferences, learned rewards, or externally verifiable outcomes. The central question is not simply whether an objective improves its own score, but what behavior that objective rewards, what capabilities are lost as the policy moves away from its reference model, and which failures remain invisible to the training signal.

This assignment compares three preference-optimization algorithms – Direct Preference Optimization (DPO), Proximal Policy Optimization (PPO), and Group Relative Policy Optimization (GRPO) – and then changes the source of feedback itself through Reinforcement Learning with Verifiable Rewards (RLVR) and Reinforcement Learning from AI Feedback (RLAIF). Training is intentionally bounded. DPO is trained from the common starting policy, whereas PPO and GRPO begin from supplied continuation checkpoints so that the assignment can emphasize controlled interventions, ablations, and failure analysis.

Task 1: Direct Preference Optimization. Offline preference optimization, regularization strength, and dataset-induced length bias.

Task 2: Proximal Policy Optimization. Online RLHF, clipping, critic behavior, and the trade-off between reward maximization and reference-policy drift.

Task 3: Group Relative Policy Optimization. Critic-free online optimization, group informativeness, and sequence-length normalization.

Task 4: Safety Calibration. Compare SFT, DPO, PPO, and GRPO on safe and unsafe prompts, including exaggerated refusal.

Task 5: RLVR versus RLAIF. Hold the policy family and group-based RL procedure approximately fixed while changing the source of feedback from exact verification to AI-generated preferences.

Task 6: Synthesis. Connect optimization mechanics, reward source, safety behavior, policy drift, and compute.

Across all tasks, use controlled comparisons, report aggregate results alongside informative failure cases, and explain why the observed behavior is plausible. A larger reward, lower KL, or higher preference accuracy is evidence about a particular metric; none is by itself a complete definition of model quality.

Common Setup

Course starter repository: <https://github.com/AbDu11aHHH/ATML-PA2-LLM-PostTraining>

Course asset repository: <https://huggingface.co/datasets/AbDu11aHHH/ATML-PA2-assets>

Clone the starter repository, install the pinned environment, download the fixed course assets, and validate the installation before beginning:

```
git clone https://github.com/AbDu11aHHH/ATML-PA2-LLM-PostTraining.git
cd ATML-PA2-LLM-PostTraining
python -m pip install -r requirements.txt
python -m scripts.download_assets
python -m scripts.validate_assets
```

The GitHub repository contains the starter code, configurations, and utilities. Large course-created checkpoints, cached diagnostics, and fixed datasets are hosted separately in the Hugging Face asset repository above. You do **not** need to clone the Hugging Face repository manually: `python -m scripts.download_assets` downloads the pinned course release into the expected local directories. Run `python -m scripts.validate_assets` afterward; the setup is complete when validation passes. Public Hugging Face base/reward/judge models are downloaded at runtime.

Use the released environment, fixed data indices, prompt pools, and model artifacts. Unless a task explicitly changes a setting, use Qwen/Qwen2.5-1.5B-Instruct as the policy family, LoRA adapters for trainable policy parameters, the released UltraFeedback preference/prompt splits, and the fixed course evaluation sets. Exact implementation constants are provided in the released configuration files and scripts; this manual focuses on the scientific comparison.

For all training and evaluation runs, keep the prompt set, decoding configuration, maximum generation length, seed, and evaluation procedure fixed across the conditions being compared. Do not choose a final setting after inspecting held-out results unless a task explicitly asks you to study that setting.

Released Code Organization

The course release uses task-specific Python scripts. Keep the task boundaries explicit and place genuinely shared functionality in `common/`. A suggested structure is shown below; filenames in the release may differ slightly, but the same separation of concerns should be preserved.

```
pa2-llm-posttraining/
  README.md
  requirements.txt
  configs/
  common/
  scripts/
    download_assets.py      # downloads fixed course assets from Hugging Face
    validate_assets.py

  task1_dpo/               # DPO objective + incomplete train/ablation scaffold
  task2_ppo/               # PPO helpers + incomplete continuation/ablation scaffold
  task3_grpo/              # GRPO helpers + incomplete continuation/ablation scaffold
  task4_safety/            # fixed judge/generation utilities; you implement evaluation
  task5_feedback/          # fixed verifier/judge utilities; you implement evaluation

  data/                   # fixed course data; most installed by download_assets
  cached/                 # supplied PPO/GRPO diagnostic caches; do not alter/commit
  checkpoints/             # supplied course checkpoints; do not alter/commit
  outputs/                 # your trained adapters/checkpoints; ignored by Git
  results/                 # your JSON/CSV logs, metrics, generated responses
```

Each task should be runnable without executing previous tasks in an interactive notebook. Save intermediate generations and metrics to disk when later analyses reuse them; do not rely on hidden notebook state.

The large checkpoint/data bundle is downloaded from <https://huggingface.co/datasets/AbDu11aHHH/ATML-PA2-assets> with `python -m scripts.download_assets`. Two tiny fixed files are handled outside that bundle: the common Task 1 word-limit prompts are tracked directly in the repository, and the Task 5 transfer file is materialized deterministically

from the first 100 examples in the official SVAMP challenge-set order if it is absent. Do not change these fixed evaluation files.

Common Metric Definitions and Reporting Conventions

Use the released metric helpers and the definitions below consistently across tasks. Do not change averaging conventions between conditions. Unless a task says otherwise, compute each metric on the same fixed examples/prompts and decoding settings for every condition being compared.

- **DPO preference accuracy.** For a held-out pair (x, y^+, y^-) , define the DPO preference margin

$$m_\theta = [\log \pi_\theta(y^+ | x) - \log \pi_{\text{ref}}(y^+ | x)] - [\log \pi_\theta(y^- | x) - \log \pi_{\text{ref}}(y^- | x)].$$

Preference accuracy is the fraction of held-out pairs with $m_\theta > 0$. Sequence log-probabilities are summed over response tokens only.

- **KL from the reference policy.** Use the released sampled-response estimator, i.e. the response-token log-probability difference between the evaluated policy and the frozen reference policy, aggregated using the course helper. Report the same sequence-versus-token averaging convention for every condition; do not replace it with a different KL implementation in one ablation.
- **Reward-model score.** Mean scalar score assigned by the fixed course reward model to generated responses. This score is useful for controlled within-task comparisons, but its numerical scale is not a calibrated measure of overall response quality and must not be compared directly with Task 5 binary or pairwise rewards.
- **PPO clip fraction.** The fraction of valid response tokens for which the importance ratio ρ_t lies outside $[1 - \epsilon, 1 + \epsilon]$ before clipping. In the cached-rollout study, also report the corresponding affected-token fraction using the same response mask.
- **Entropy.** Mean token-level policy entropy over valid generated response tokens. Interpret it as a diversity/confidence diagnostic, not as a quality metric.
- **GRPO informative-group rate.** A prompt group is informative when the sampled rewards have non-zero standard deviation under the numerical tolerance used by the released helper. Report both mean within-group reward standard deviation and the fraction of zero-standard-deviation (uninformative) groups.
- **Exact-answer accuracy and format compliance.** Exact-answer accuracy is the fraction of examples for which the released parser extracts the designated final answer and it matches the gold answer. Format compliance is the fraction for which the required final-answer format is successfully parsed, irrespective of correctness.
- **AI pairwise win rate.** For pairwise judge comparisons, count a win as 1, a tie as 0.5, and a loss as 0, then average over the fixed comparison set. Report the number of judge ties or ambiguous outputs separately when requested.
- **Response length.** Number of generated response tokens after the prompt, excluding padding. When a task asks for length statistics, report at least the mean and one dispersion statistic (standard deviation or interquartile range) using the same tokenizer and generation cap.

Evidence requirements. The “Required Evidence” list in each task specifies the diagnostics, comparisons, and qualitative evidence that must be present in the main 8-page report. The choice of how to organize these results into figures and tables is yours. You may combine related diagnostics in a single figure/table or separate them when that improves clarity, provided all required quantities are reported and the presentation remains readable within the page limit.

1 Direct Preference Optimization (DPO)

Introduction and Terminology

Direct Preference Optimization learns from a fixed dataset of preference pairs without first fitting an explicit reward model and without collecting on-policy rollouts. For each prompt x , the dataset supplies a preferred completion y^+

and a rejected completion y^- . DPO compares how the trainable policy changes the relative likelihood of these two responses with respect to a frozen reference policy:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E} \left[\log \sigma \left(\beta \left[\log \frac{\pi_{\theta}(y^+ | x)}{\pi_{\text{ref}}(y^+ | x)} - \log \frac{\pi_{\theta}(y^- | x)}{\pi_{\text{ref}}(y^- | x)} \right] \right) \right].$$

The sequence log-probability is the sum of response-token log-probabilities under teacher forcing. The coefficient β controls the scale of the preference log-ratio relative to the reference. Because sequence likelihood accumulates over tokens, response length and the length structure of the preference dataset can interact with optimization.

Readings

Required

- **Rafailov et al. (2023)** – *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. Focus on the derivation of the DPO objective and the role of the reference policy.

Optional

- **Hugging Face TRL** – *DPO Trainer documentation*. Useful for implementation conventions and terminology; the assignment equations and released configuration define the required experiment.
- **Medium (Matthew Gunton)** – *Understanding Direct Preference Optimization*. Connects the DPO objective to the earlier PPO/RLHF pipeline and discusses the preference log-ratio interpretation and consequences of the formulation.
- **(Luis Serrano Academy)** – *Direct Preference Optimization (DPO) – How to fine-tune LLMs directly without reinforcement learning*. Covers RLHF versus DPO, Bradley–Terry preference modeling, KL regularization, and the DPO loss.

Dataset, Models, and Experimental Setup

Use the course-provided UltraFeedback preference files. The standard and length-controlled conditions use fixed course subsets, and every DPO condition begins from the same Qwen2.5-1.5B-Instruct initialization with the supplied LoRA configuration. The starter provides the data/model plumbing and DPO objective scaffold, but you must complete the training loop and validate the objective implementation before use. Train the standard DPO model for one epoch.

A separate course-provided length-control file is used only for the third DPO experiment. It contains equal numbers of pairs from three strata: (i) the preferred response is clearly longer than the rejected response, (ii) the two responses are approximately length matched, and (iii) the rejected response is clearly longer than the preferred response. You do not construct or rebalance this dataset yourself.

Steps

1. **Standard DPO.** Validate the released implementation, train for one epoch on the fixed preference set, and evaluate on the fixed held-out pairs. Report DPO loss, held-out preference accuracy, KL from the reference policy, reward-model score on generated responses, and response-length statistics. This run establishes the common DPO baseline; the key question is whether stronger preference fitting is accompanied by useful behavior rather than disproportionate policy drift or a trivial length shift.
2. **Regularization-Strength Study.** From the original policy initialization, train the released short-run DPO configuration with $\beta \in \{0.03, 0.10, 0.30\}$. Keep the data subset, number of examples, optimizer, seed, and LoRA configuration fixed. Compare preference fitting, KL, reward score, and generated length. Because only β changes, this isolates how the DPO preference scale changes the fit-versus-reference-drift trade-off over the tested range.
3. **Length-Confounding Study.** Train one additional DPO model from the original initialization on the supplied length-balanced training subset. Evaluate both the standard DPO model and the length-balanced DPO model on the supplied length-stratified held-out set, reporting results separately for preferred-longer, matched-length, and rejected-longer examples. Compare generated response length and explicit word-limit compliance on the common prompt set. This tests whether apparent preference gains and verbosity are partly explained by length correlations in the training pairs rather than by content quality alone.

What to Watch For

- Preference accuracy, reward score, KL, and response length measure different properties. Do not treat one metric as a complete measure of alignment quality.
- A length correlation in the preference data is a property of the dataset; a length shift in generated outputs is a property of the learned policy. Separate these observations in your discussion.
- The balanced length-control set is a diagnostic intervention, not a replacement for the original preference distribution.
- When comparing β values, use identical held-out prompts and decoding settings.

Required Evidence

- **DPO summary results:** report standard DPO and the three short-run β conditions with held-out DPO loss/preference accuracy, KL, reward-model score, and response-length statistics. Clearly indicate that the standard one-epoch run and the short β forks use different budgets.
- **Length-confounding diagnostics:** compare standard versus length-balanced DPO on preference accuracy for the preferred-longer, matched-length, and rejected-longer strata, together with generated response length and word-limit compliance on the common prompt set.
- **Qualitative evidence:** include brief examples that illustrate (i) a preference/reward-versus-quality disagreement and (ii) behavior relevant to length bias or instruction compliance. Quote only the minimum text needed to support the point.

Research Questions

1. How does changing β alter preference fitting, KL from the reference policy, and reward-model score? Is the relationship monotonic over the tested range?
2. How much of the observed length behavior can be explained by the preference dataset itself? Does balancing the three length strata materially change which examples are learned well or how long the model responds?
3. Find cases in which the aligned policy has a stronger preference/reward signal but a worse response according to correctness, concision, or instruction compliance. What does this reveal about offline preference optimization?

2 Proximal Policy Optimization (PPO)

Introduction and Terminology

PPO treats language generation as an online reinforcement-learning problem. The current policy generates responses, a frozen reward model assigns a scalar reward, and a value model estimates expected future return. PPO updates the policy using trajectories sampled from an older policy snapshot while restricting how far the new policy can move in one update. The token-level probability ratio is

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)},$$

and the clipped surrogate objective is

$$L_{\text{clip}}(\theta) = \mathbb{E}_t[\min(\rho_t A_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) A_t)].$$

Credit assignment is obtained through a value function. With temporal-difference residual δ_t and GAE parameter λ ,

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t), \quad A_t^{\text{GAE}} = \sum_{k \geq 0} (\gamma \lambda)^k \delta_{t+k}.$$

In RLHF, the terminal learned reward is typically supplemented by a reference-policy penalty. A sampled-token approximation to KL shaping is

$$r_t = r_{\text{task}} \mathbf{1}[t = T] - \beta_{\text{KL}} (\log \pi_{\theta}(a_t | s_t) - \log \pi_{\text{ref}}(a_t | s_t)).$$

The clipping parameter ϵ controls how aggressively a single batch may change token probabilities, whereas β_{KL} controls longer-horizon pressure to remain near the reference policy. The two mechanisms are related but not interchangeable.

Readings

Required

- Schulman et al. (2017) – *Proximal Policy Optimization Algorithms*. Focus on the clipped surrogate objective.
- Ouyang et al. (2022) – *Training Language Models to Follow Instructions with Human Feedback*. Focus on the RLHF pipeline, reward modeling, and reference-model regularization.

Optional

- Hugging Face Blog – *The N Implementation Details of RLHF with PPO*. Reproduces the original RLHF–PPO pipeline and catalogs practical details around reward/value computation, KL shaping, whitening, sampling temperature, and optimization stability.
- Medium (Cameron R. Wolfe) – *Proximal Policy Optimization (PPO): The Key to LLM Alignment*. Develops policy gradients, actor–critic/value estimation, PPO clipping, and the way these components fit into language-model alignment.
- (Julia Turc) – *Proximal Policy Optimization (PPO) for LLMs Explained Intuitively*. Builds from policy gradients and actor–critic methods to GAE, importance sampling, and PPO clipping in the LLM setting.

Dataset, Models, and Experimental Setup

Load the supplied PPO continuation bundle, which contains a policy LoRA checkpoint, the matched value-model state, the frozen reference configuration, the reward-model identifier, fixed prompt IDs, and cached diagnostic rollouts. Continue from the supplied checkpoint rather than restarting PPO. The starter provides checkpoint/model loading and PPO objective utilities, but you must complete the continuation loop and ablation orchestration and validate the objective implementation. Use the release configuration for rollout generation and optimization.

Steps

1. **Standard PPO Continuation.** Validate the implementation and run the required 20-update continuation from the supplied checkpoint. Log mean learned reward, KL from the reference, policy loss, value loss, entropy, gradient norm, clip fraction, and response length. This establishes the local behavior of the supplied PPO midpoint under the reference configuration and lets you check whether reward improvement, policy drift, and critic behavior evolve consistently.
2. **Clipping Study.** Use the supplied cached rollout batch to measure the clipped surrogate and affected-token fraction for $\epsilon \in \{0.05, 0.20, 0.50\}$. Then run the short fixed-budget continuation forks specified in the release from the same PPO checkpoint. Compare optimization stability and final held-out behavior under identical reward/KL settings. Changing ϵ modifies the size of policy-ratio updates that are allowed before clipping; the cached batch exposes the immediate geometric effect, while the matched short forks test whether that difference translates into different optimization behavior.
3. **Reward-Overoptimization Study.** From the same PPO checkpoint, compare $\beta_{\text{KL}} \in \{0, 0.10, 0.20\}$ under the fixed short continuation budget. Track how learned reward, KL, entropy, response length, and held-out qualitative quality change. Inspect cases where reward increases without a corresponding improvement in the response. β_{KL} directly changes the cost of leaving the reference policy, so this study asks whether additional learned reward is purchased with larger drift and whether that trade-off remains aligned with held-out response quality.

What to Watch For

- Clipping affects the optimization geometry of a batch; KL pressure constrains drift from a separate reference policy. Interpret the two controls separately.
- A reward increase can reflect useful preference learning, harmless specialization, or exploitation of the reward model. Use held-out examples and the safety task before deciding which explanation is plausible.
- The value model introduces an additional learned component. Report its behavior rather than assuming the critic is accurate because policy reward improved.
- Compare short forks at equal generated-token and update budgets.

Required Evidence

- **Standard PPO continuation diagnostics:** report the trajectories of reward and KL, policy/value losses, entropy, clip fraction, gradient norm, and response length over the continuation. Also report peak VRAM and wall-clock time for the standard continuation.
- **Clipping study:** for $\epsilon \in \{0.05, 0.20, 0.50\}$, report cached-rollout clip/affected-token fraction and the matched short-fork held-out reward, KL, response length, and at least one clearly defined stability statistic.
- **KL-pressure study:** for $\beta_{\text{KL}} \in \{0, 0.10, 0.20\}$, report held-out reward, KL, entropy, and response length.
- **Qualitative evidence:** include brief examples showing at least one case where reward and response quality move together and at least one where they disagree.

Research Questions

1. How does ϵ change the fraction of policy updates constrained by clipping and the stability of the short continuation?
2. When KL pressure is weakened, which observable changes first: reward, policy drift, entropy, response length, or qualitative behavior? Use logged trajectories to support your answer.
3. How informative is the learned reward as the policy moves away from the reference distribution? Identify evidence for or against reward overoptimization in your run.

3 Group Relative Policy Optimization (GRPO)

Introduction and Terminology

GRPO removes PPO’s learned critic and instead compares multiple responses sampled for the same prompt. For a prompt x with K completions and rewards $r_1, \dots, r_K$, a group-relative advantage can be written as

$$A_k = \frac{r_k - \mu_r}{\sigma_r + \varepsilon}, \quad \mu_r = \frac{1}{K} \sum_{j=1}^K r_j.$$

A PPO-style clipped likelihood-ratio objective is then applied to the sampled completions:

$$\mathcal{L}_{\text{GRPO}} = -\mathbb{E}_k \left[\frac{1}{T_k} \sum_{t=1}^{T_k} \min(\rho_{k,t} A_k, \text{clip}(\rho_{k,t}, 1 - \epsilon, 1 + \epsilon) A_k) \right] + \beta D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}}).$$

Removing the critic reduces one source of memory and estimation error, but the usefulness of an update now depends strongly on reward variation inside each prompt group. Recent work has also examined the $1/T_k$ sequence normalization because it changes how short and long completions contribute to the gradient. The course release includes a Dr. GRPO-style normalization condition for a controlled comparison.

Readings

Required

- **Shao et al. (2024)** – *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models*. Focus on the GRPO objective and critic-free motivation.
- **Liu et al. (2025)** – *Understanding R1-Zero-Like Training: A Critical Perspective*. Focus on the optimization biases identified in GRPO and the Dr. GRPO modification.

Optional

- **DeepSeek-AI et al. (2025)** – *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. Useful for broader context on reasoning-oriented RL and verifiable rewards.
- **Medium (Astarag Mohapatra)** – *PPO to GRPO in Large Language Models Alignment*. Connects PPO to GRPO, explains group-relative advantages, and contrasts critic-based PPO with critic-free group-relative optimization.
- **(Julia Turc)** – *DeepSeek’s GRPO (Group Relative Policy Optimization)*. Builds from policy gradients, REINFORCE, actor-critic methods, and PPO to the within-group comparison used by GRPO.

Dataset, Models, and Experimental Setup

Load the supplied GRPO midpoint adapter and the same policy family, reward model, reference policy, and Ultra-Feedback prompt pool used for PPO. The baseline continuation uses $K = 4$ sampled completions per prompt with max-length completions masked from the training loss. The starter provides checkpoint/model loading and GRPO objective utilities, but you must complete the continuation loop and ablation orchestration and validate the objective implementation. The course also supplies a fixed completion/reward cache for the group-size diagnostic so that K can be studied without regenerating three complete rollout sets.

Steps

1. **Standard GRPO Continuation.** Validate the released implementation and run the required 20-update continuation. Log reward, KL, within-group reward standard deviation, fraction of uninformative groups, policy loss, gradient norm, entropy, and response length. This establishes the critic-free baseline and shows whether useful learning signal is consistently available across prompt groups rather than only in aggregate reward.
2. **Group-Size Study.** Using the supplied completion/reward cache, regroup the same fixed generation budget into $K \in \{2, 4, 8\}$. Compare the fraction of informative groups, variance of the group-relative signal, and sensitivity to prompt difficulty. No additional policy training is required for this study. Increasing K gives more within-prompt comparisons but spends more samples on each prompt; holding total generations fixed isolates whether larger groups actually make useful relative advantages more likely.
3. **Length-Normalization Study.** From the same midpoint checkpoint, run matched short continuations using canonical GRPO sequence normalization and the supplied Dr. GRPO-style normalization. Keep reward, β , ϵ , prompts, generation settings, and generated-token budget fixed. Compare reward, KL, response length, length-conditioned gradient/statistics, and held-out quality. The reward function is unchanged, so any systematic shift in gradient allocation or response length can be attributed to how token/sequence contributions are normalized rather than to a different feedback signal.

What to Watch For

- Larger K consumes more generations per prompt; compare group-size conditions at equal total generations rather than equal numbers of prompts.
- Binary or low-resolution rewards can produce many uninformative groups for both very easy and very difficult prompts.
- Length normalization can alter optimization even when the reward function is unchanged. Analyze the change in both gradient statistics and generated behavior.

- Removing the critic reduces one learned component, but does not make the relative reward signal noiseless.

Required Evidence

- **Standard GRPO continuation diagnostics:** report reward and KL; mean within-group reward standard deviation and uninformative-group fraction; policy loss, gradient norm, entropy; and response length over the continuation. Also report peak VRAM and wall-clock time for the standard continuation.
- **Equal-generation group-size study:** for $K = 2, 4, 8$, report informative-group rate, mean within-group reward standard deviation, variance of the group-relative signal, and the same quantities for at least two prompt-difficulty bins. Define the binning rule once.
- **Canonical versus Dr.-GRPO comparison:** report held-out reward, KL, response length, and at least one length-conditioned gradient/statistic that directly tests the normalization effect.
- **Qualitative evidence:** include brief examples that help interpret the normalization or group-informativeness results.

Research Questions

1. How does group size change the probability of observing a useful relative signal, and which prompt-difficulty regimes benefit most from larger K ?
2. Does changing the sequence normalization alter response length or the allocation of gradient magnitude across short and long completions? Are those changes associated with quality?
3. After removing the critic, what becomes the dominant source of instability or sample inefficiency in your observed GRPO run?

4 Safety Calibration of the Aligned Policies

Introduction and Terminology

Preference optimization can change refusal behavior even when safety was not the explicit training objective. A useful aligned model should avoid harmful compliance while still answering benign requests that merely contain sensitive vocabulary. This task therefore treats safety as a calibration problem rather than a one-dimensional refusal score.

XSTest contains safe prompts and related unsafe contrasts designed to expose exaggerated safety behavior. Evaluate the four frozen policies defined below only after the earlier alignment experiments are complete; Task 4 results may not be used to retune Tasks 1–3. In particular, do not select an ablation checkpoint because it looks better on XSTest.

Readings

Required

- Röttger et al. (2023) – *XSTest: A Test Suite for Identifying Exaggerated Safety Behaviours in Large Language Models*. Focus on the distinction between unsafe compliance and exaggerated refusal on safe prompts.

Optional

- Bianchi et al. (2023) – *Safety-Tuned LLaMAs: Lessons From Improving the Safety of Large Language Models that Follow Instructions*. Useful background on the helpfulness–harmlessness trade-off and exaggerated safety after safety tuning.
- Hugging Face XSTest dataset card – *XSTest dataset overview*. Summarizes the safe and unsafe contrast prompts and the benchmark’s goal of measuring exaggerated refusal alongside harmful compliance.
- Anthropic – *Challenges in evaluating AI systems*. Discusses evaluation limitations and the tension between helpfulness and harmlessness, providing context for interpreting refusal/compliance metrics and judge agreement.

- **Medium (Quaxel)** – *When Refusal Tuning Backfires on Harmless Prompts*. Describes refusal-tuning failure modes that cause harmless prompts to be rejected and frames safety evaluation around calibrated refusal rather than refusal rate alone.

Dataset, Models, and Experimental Setup

Use the fixed XSTest CSV in the course release. The four policies for this task are fixed as follows:

- **SFT baseline:** the untouched Qwen/Qwen2.5-1.5B-Instruct starting policy with no Task 1–3 adapter applied;
- **DPO:** the final adapter from **Task 1, Step 1 (Standard DPO)**, after the required one-epoch training run;
- **PPO:** the final policy adapter from **Task 2, Step 1 (Standard PPO Continuation)**, after the required 20 updates;
- **GRPO:** the final policy adapter from **Task 3, Step 1 (Standard GRPO Continuation)**, after the required 20 updates.

Do **not** substitute a β -sweep DPO model, PPO clipping/KL ablation fork, or GRPO normalization fork for these standard-policy checkpoints. Generate one deterministic response per policy and prompt. The starter supplies the frozen AI judge loader, prompt, parser, and single-example scoring function; you must implement the evaluation orchestration and requested aggregate analysis. The released AI scorer is a categorical judge, not a scalar reward model. For each prompt–response pair it returns one behavior label from

{SAFE_ANSWER, JUSTIFIED_REFUSAL, UNSAFE_COMPLIANCE, OVER_REFUSAL, AMBIGUOUS},

together with a confidence field used only for auditing. The complete judge prompt and parser are supplied in the evaluation scripts.

Steps

1. **Common Safety Evaluation.** Generate responses from the four fixed policies above for every XSTest prompt using identical deterministic decoding and the common maximum response length. Freezing both the model choice and decoding procedure makes this a post-hoc calibration comparison rather than another model-selection stage.
2. **Categorical AI Scoring.** Run the fixed AI behavior judge. Report safe-prompt answer rate, safe-prompt over-refusal rate, unsafe-prompt unsafe-compliance rate, unsafe-prompt justified-refusal rate, ambiguous-judge rate, category-level results, and response length. These paired safe/unsafe rates separate genuine reduction in harmful compliance from indiscriminate refusal, which a single refusal metric would conflate.
3. **Manual Audit.** Manually audit the fixed 60-example subset, balanced across safe/unsafe categories. Assign the same five behavior labels without looking at the AI label first. Report automated-versus-manual agreement and analyze policy disagreements and judge errors. The manual labels estimate the reliability and failure modes of the AI judge itself, so disagreements should be used to qualify the automated policy comparison rather than silently discarded.

What to Watch For

- An increase in unsafe refusal can coexist with an increase in safe over-refusal. Report both sides of the trade-off.
- The AI judge is an evaluation instrument, not ground truth. Use the manual audit to characterize its failure modes.
- Safety differences across DPO, PPO, and GRPO are observational because their learned policies differ in more than one training detail. Avoid attributing a behavior to an optimizer alone unless the comparison isolates that factor.

Required Evidence

- **Safety-calibration comparison:** for each fixed policy (SFT, standard DPO, standard PPO, standard GRPO), report safe-answer rate, safe over-refusal rate, unsafe-compliance rate, justified-refusal rate, ambiguous-judge rate, and mean response length.

- **Category-level behavior:** report the judge-label distribution across the XSTest categories for all four policies so that category-specific failure modes are visible.
- **Manual-audit agreement:** on the fixed 60 manually audited examples, report judge/manual agreement, including a confusion-style breakdown and ambiguous-rate information.
- **Qualitative disagreement evidence:** include brief examples covering harmful compliance, justified refusal, and exaggerated refusal/over-refusal. For each, state whether the disagreement reflects a policy difference, a judge error, or both.

Research Questions

1. Do higher preference/reward scores in Tasks 1–3 correspond to better safety calibration, or are there counterexamples?
2. Which policies most often confuse sensitive wording with harmful intent, and what training signals could plausibly contribute to that pattern?
3. What kinds of responses does the AI safety judge misclassify, and how much do those errors affect the policy comparison?

5 Feedback Source: RLVR versus RLAIIF

Introduction and Terminology

The preceding tasks change the policy-optimization algorithm while relying on preference-based feedback. This task instead holds the policy family and group-based RL procedure approximately fixed and changes the source of feedback. **Reinforcement Learning with Verifiable Rewards (RLVR)** uses an externally checkable outcome, whereas **Reinforcement Learning from AI Feedback (RLAIIF)** uses an AI model as the feedback provider.

For a problem x with response y , the course RLVR condition uses an exact final-answer checker:

$$r_{\text{RLVR}}(x, y) = \mathbf{1}[\text{verifier}(y) = \text{gold}(x)].$$

For GSM8K, this verifier extracts the designated final number and assigns a binary reward. The reward is reproducible and exact with respect to the checker, but it does not distinguish two wrong solutions and it gives no credit for a correct reasoning path whose designated final answer is wrong.

Canonical RLAIIF commonly uses an AI model to create preference labels that can train a reward model. The supplied checkpoint instead uses a direct-RLAIIF-style group reward. For the K responses sampled for one prompt, a frozen AI judge makes pairwise preference decisions. Each response receives the normalized pairwise win rate

$$r_{\text{RLAIIF}}(y_k) = \frac{\text{wins}(y_k) + 0.5 \text{ ties}(y_k)}{K - 1}.$$

The resulting value lies in $[0, 1]$ and is used by the same group-based policy optimization machinery. This makes the comparison interpretable: RLVR supplies exact but sparse outcome information, whereas RLAIIF can express relative differences in response quality but inherits judge noise, style preferences, and inference cost.

Readings

Required

- **DeepSeek-AI et al. (2025)** – *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. Focus on rule-based/verifiable rewards and reasoning-oriented RL.
- **Lee et al. (2024)** – *RLAIIF vs. RLHF: Scaling Reinforcement Learning from Human Feedback with AI Feedback*. Focus on AI-generated preferences and direct RLAIIF.

Optional

- **Bai et al. (2022)** – *Constitutional AI: Harmlessness from AI Feedback*. Useful broader context on replacing some human supervision with model-generated feedback.
- **Medium (Adnan Masood)** – *Reinforcement Learning with Verifiable Rewards: Definitions, Methods, Case Studies, and Evaluation Caveats*. Covers verifier-based rewards, outcome-only RL, reward hacking, case studies, and evaluation caveats for RLVR systems.
- **(Adam Lucek)** – *Reinforcement Learning with Verifiable Rewards – Teaching LLMs to Solve Problems*. Walks through the LLM training lifecycle, verifier/reward construction, an RLVR environment, and a worked training setup.
- **Medium (Quaxel)** – *RLHF vs RLAIF: Feedback Is the New Data*. Contrasts human- and AI-generated preference pipelines and discusses scaling, evaluator bias, and the reliability of the resulting feedback signal.
- **(AI Makerspace)** – *Reinforcement Learning with AI Feedback (RLAIF) / Constitutional AI*. Walks through critique-and-revision, AI preference ranking, and the reinforcement-learning stage of an RLAIF pipeline.

Dataset, Models, and Experimental Setup

Two course-supplied LoRA adapters are used. Both start from the same Qwen2.5-1.5B-Instruct policy and are prepared on the same fixed GSM8K prompt indices using the same group-based optimizer family, group size, maximum completion length, and approximately matched generated-token budget. The starter supplies the exact final-answer verifier, pairwise AI judge, policy/data loaders, and diagnostic-set parser; you must implement the requested evaluation and analysis. The RLVR adapter is trained with the exact final-answer verifier. The RLAIF adapter is trained using pairwise preferences from the frozen AI judge.

For the out-of-domain comparison, the course uses a fixed 100-example SVAMP subset. The release tooling deterministically selects the first 100 examples in the official SVAMP challenge-set order and stores them as `data/math_transfer_eval.jsonl`. You must use that file unchanged.

The course also supplies a fixed **Controlled Reward Diagnostic Set**. It contains 100 manually validated responses built from 20 GSM8K problems, with five variants per problem:

- clean reasoning with a correct final answer;
- a correct final answer with corrupted intermediate reasoning;
- coherent or mostly correct reasoning with an incorrect designated final answer;
- a correct response augmented with irrelevant persuasive filler;
- a response that mentions the gold number as a distractor but ends with an incorrect designated final answer.

The diagnostic set is designed to expose cases where exact verification and AI preference feedback disagree.

Steps

1. **In-Domain Comparison.** Evaluate SFT, RLVR, and RLAIF on the fixed GSM8K evaluation subset. Report exact final-answer accuracy, RLAIF pairwise win rate against SFT under the fixed judge, format compliance, response length, and agreement between the exact verifier and AI preference signal. This measures the end behavior obtained when the policy family and group-based RL procedure are similar but the feedback source changes from exact outcome verification to AI preferences.
2. **Reward-Sensitivity and Robustness Study.** Score the supplied 100-response diagnostic set using both the exact verifier and the AI pairwise judge. For each controlled pair, report the fraction for which the mechanism prefers the diagnostically better response, ties, and incorrect preferences. Break results down by perturbation type. In particular, separate *reasoning sensitivity* – cases where the final answer is held correct while reasoning is degraded – from *outcome sensitivity* – cases where the reasoning is held approximately fixed while the designated final answer is changed. The controlled perturbations isolate what each feedback mechanism can and cannot detect, distinguishing outcome blindness from judge sensitivity to reasoning, verbosity, or persuasive style.
3. **Out-of-Domain Comparison.** Evaluate the same three policies on the fixed course transfer set without retraining. Use exact answer accuracy and the same AI pairwise evaluation protocol. Compare performance drop, response

length, and failure types. This tests whether behavior learned under either feedback source transfers beyond the training-domain prompt distribution rather than merely fitting GSM8K-specific patterns.

For the diagnostic pairs, define

$$S_{\text{reason}} = \Pr[R(y_{\text{clean}}) > R(y_{\text{reason-corrupt}})],$$

for pairs with the same correct final answer, and

$$S_{\text{outcome}} = \Pr[R(y_{\text{correct final}}) > R(y_{\text{wrong final}})]$$

for outcome-changing pairs. Also report tie and wrong-preference rates. A binary verifier may legitimately tie on reasoning-only changes; that invariance is itself part of the result rather than an “accuracy error.”

What to Watch For

- RLVR is only as correct as the verifier specification. A robust final-answer parser should not reward a gold number that appears only as a discarded intermediate candidate.
- RLAIF can distinguish degrees or styles of response quality, but the preference label is generated by another model and can reflect judge-specific biases.
- Do not compare the numerical scale of binary RLVR reward and pairwise RLAIF win rate as if they were calibrated utilities. Compare correctness, ranking, disagreement patterns, and robustness.
- A high reasoning-sensitivity value is useful only if the judge is reacting to reasoning quality rather than verbosity, confidence, or surface style. Use the filler condition to test this possibility.

Required Evidence

- **In-domain comparison:** evaluate SFT, RLVR, and RLAIF on the fixed GSM8K subset and report exact accuracy, AI pairwise result, format compliance, mean response length, and verifier–judge agreement.
- **Controlled diagnostic study:** for all five supplied perturbation categories and for each feedback mechanism, report better-response rate, tie rate, and wrong-preference rate. Also report S_{reason} and S_{outcome} .
- **Out-of-domain comparison:** evaluate the same three policies on the fixed SVAMP subset and report exact accuracy, AI pairwise result, response length, and the drop from the corresponding in-domain metric where applicable.
- **Qualitative diagnostic evidence:** include brief examples spanning reasoning-only corruption, persuasive filler, and wrong-final/gold-distractor behavior to explain verifier–judge disagreement.
- **Feedback-source discussion:** briefly compare feedback coverage, noise, exploitability, and inference cost using the quantitative and qualitative evidence above.

Research Questions

1. Where does AI preference feedback distinguish responses that the binary verifier treats identically, and when is that extra flexibility useful?
2. When does the AI judge make an incorrect distinction because of style, verbosity, or persuasive framing? How does this change the interpretation of its reasoning sensitivity?
3. Which reward source is more vulnerable to each diagnostic category, and what does the pattern imply about verifier specification versus judge bias?
4. Which behaviors transfer to the out-of-domain set, and what evidence supports or weakens the claim that either feedback source promotes more general reasoning behavior?

6 Synthesis of Results & Report Writing Guidelines

Treat the assignment as one study of post-training rather than five disconnected exercises. The unifying question is: **how do optimization constraints and reward sources shape what a language model learns to optimize, and when do the measured objectives fail to match the behavior we actually want?**

Cross-task Analysis: No new training runs are required for Task 6. Synthesize the evidence already reported in Tasks 1–5. You may include a compact cross-task summary if it helps the discussion, but avoid repeating task-specific results without adding new interpretation.

- **Preference strength versus policy drift.** Connect DPO β , PPO KL pressure, and GRPO normalization to KL, entropy, reward, and response length.
- **Offline versus online feedback.** Contrast DPO’s fixed preference pairs with PPO/GRPO learning on newly sampled outputs.
- **Optimization bias.** Connect DPO length confounding, PPO clipping/reward overoptimization, and GRPO group/length effects.
- **Safety calibration.** Determine whether better preference/reward metrics correspond to lower unsafe compliance without excessive refusal.
- **Reward-source design.** Compare learned preference reward, exact verification, and AI feedback in terms of coverage, noise, exploitability, and compute.

Deliverables:

- **GitHub Repository:** Python scripts, configuration files, fixed split IDs, logs, machine-readable results, and a top-level README containing exact commands to reproduce every required task. Do not submit a monolithic notebook as the experiment pipeline. Do not commit raw datasets or large model weights.
- **PDF Report:** 8 pages of main content in the course NeurIPS format. References do not count toward the page limit. A short appendix may contain implementation details or extra results, but graders are not required to read it and it cannot replace required evidence.
- **LMS Submission:** upload the PDF and submit an accessible GitHub repository link in the format announced on LMS.

Suggested Report Organization: Abstract; Introduction and research questions; Experimental Setup/Methods; Results for Tasks 1–5; Cross-task Discussion; Conclusion; References. Keep paper-method summaries concise and spend report space on controlled evidence, disagreements, failure cases, and interpretation.

Before You Submit:

- Every reported value should trace to a saved result file and a reproducible Python command.
- All policy comparisons must use the fixed prompt IDs and decoding settings specified by the task.
- Do not use Task 4 safety labels or Task 5 evaluation/diagnostic data to tune earlier policy training.
- Clearly attribute materially reused external code in the README.
- Freeze the final repository state with a meaningful commit before submitting the report.

Appendix: Git and Version Control for Research Code

This appendix is a practical add-on. The goal is to make your repository behave like a small research codebase: experiments should be recoverable, changes should be attributable, and the exact code used for the report should be identifiable.

Recommended Overall Repository Layout:

```
pa2-llm-posttraining/  
  README.md  
  requirements.txt  
  .gitignore  
  configs/  
  common/  
  task1_dpo/
```

```

task2_ppo/
task3_grpo/
task4_safety/
task5_feedback/
data/           # supplied subsets / IDs; keep raw public data out of Git
checkpoints/    # supplied/downloaded weights; ignored by Git
results/        # small JSON/CSV logs and generated-response records
report/
figures/

```

Keep task boundaries obvious. Move a helper into `common/` only when multiple tasks genuinely share it. The README should provide one command sequence per required experiment.

1. Create or clone the repository:

```

git clone <repository-url>
cd <repository>
# or for an existing local folder:
git init
git remote add origin <repository-url>

```

2. Add a `.gitignore` early:

Ignore environments, caches, downloaded datasets, and large checkpoints, but keep small CSV/JSON result files that support the report.

```

.venv/
__pycache__/
*.pyc
.cache/
wandb/
checkpoints/
*.pt
*.pth
*.safetensors

```

3. Commit coherent changes regularly:

```

git status
git add task2_ppo/ppo.py task2_ppo/continue_train.py
git commit -m "Validate PPO objective and continuation pipeline"
git push

```

A useful commit should capture one coherent change: a corrected objective, an evaluation script, a new metric, or a reproducibility improvement. Avoid making your first meaningful commit at the deadline.

4. Use branches for risky experiments:

```

git switch -c task3-normalization
# implement and test
git add .
git commit -m "Add GRPO normalization comparison"
git switch main
git merge task3-normalization

```

After a merge, rerun at least a smoke test before trusting previously generated results.

5. Useful inspection and recovery commands:

```

git log --oneline --graph --decorate -15
git diff
git diff --staged
git restore path/to/file.py
git show <commit>:task1_dpo/dpo.py

```

Do not run destructive Git commands you do not understand. If you accidentally add a large dataset/checkpoint, stop tracking it and add an appropriate pattern to `.gitignore` before pushing more copies.

6. Freeze the submitted code state:

When the final results and report are fixed, commit and push the exact scripts used to produce them.

```

git add .
git commit -m <meaningful commit message>
git push origin main

```